

Laborator 4.

Proiectarea sistemului de operare în timp real MyOS. Task-ul. Comutarea în funcție de evenimente

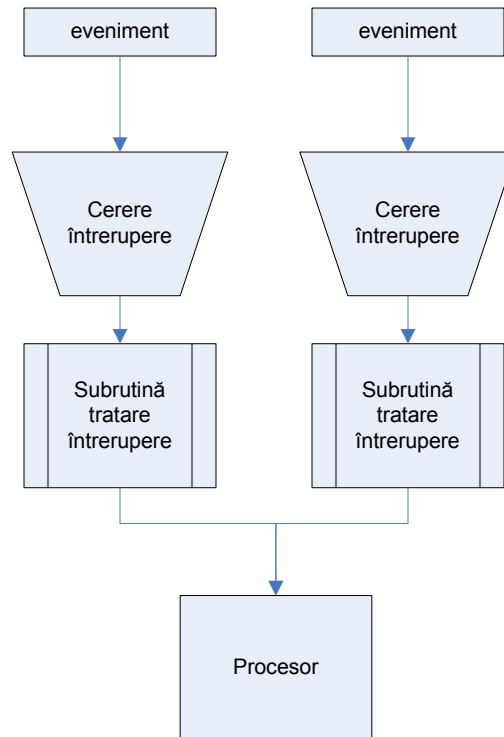
1. Obiectiv

În acest laborator sunt studiate tehnicile de comutare între procese în cadrul unui sistem în timp real. Așa cum s-a văzut în laboratorul anterior, principalul obiectiv al administratorului sistemului de operare în timp real este comutarea în timp realizată între procese (time sharing). În acest laborator se va vedea cum acest mecanism se poate folosi pentru a răspunde la evenimente.

2. Breviar teoretic

Principalul obiectiv al unui sistem în timp real este capacitatea de a răspunde în timp util la evenimente externe. O soluție pentru aceasta ar fi utilizarea întreruperilor ca metode de captare și răspuns la evenimente.

Soluția este prezentată în figura de mai jos:



Având mai multe surse de întrerupere pentru același microprocesor sau microcontroler, acesta poate fi programat să răspundă la toate aceste întreruperi atunci când ele apar. În cazul în care se dorește pentru fiecare întrerupere i se poate stabili un nivel de prioritate pentru cazul în care sunt recepționate mai multe cereri de întreruperi în același timp.

Soluția cu cereri de întrerupere este una prin care se asigură răspunsul ferm la evenimente dacă numărul acestora nu este unul prea mare.

Procedura în acest caz este următoarea:

Apare eveniment

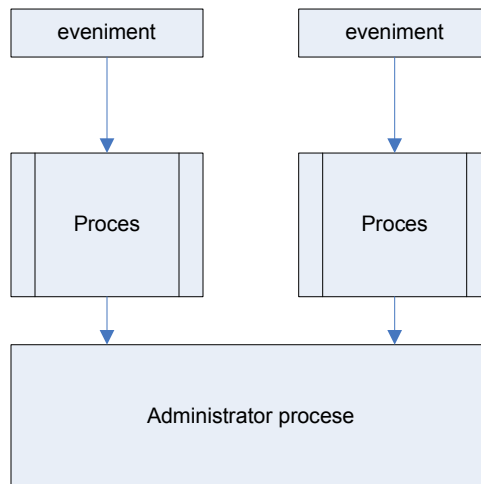
Generare cerere de întrerupere

Rulare subrutina de tratare a întreruperii

Problema cu acest tip de sistem apare atunci când numărul evenimentelor este mai mare decât când probabilitatea de apariție a două evenimente la aceleași momente de timp este mare. În acest caz, sistemul cu priorități poate să nu fie unul eficient.

De exemplu, dacă la apariția unui eveniment E1 este generată o cerere de întrerupere acesta va determina rularea unei subrutine de tratare a întreruperii S1. Să presupunem că în timpul rulării subrutinei S1 apare un eveniment E2 care necesită un timp de răspuns maxim admis (deadline time) mai mic decât durata de rulare a subrutinei S1. Există două posibilități:

1. Este rulat subrutina S1 până la capăt apoi se începe tratarea evenimentului E2. Soluția nu este în acest caz corectă deoarece tratarea evenimentului E2 se face după expirarea timpului de răspuns cerut.
2. Subrutina care tratează întreruperea declanșată de evenimentul E2 este pusă pe un grad de prioritate mai mare. Aceasta înseamnă că subrutina S1 este suspendată în momentul în care apare întreruperea declanșată de evenimentul E2 și este rulat subrutina de tratare a întreruperii S2. După ce aceasta este executată se rulează apoi în continuare S1. Nici această soluție nu poate fi corectă deoarece timpul de execuție al subrutinei S2 poate fi unul prea mare comparativ cu timpul de răspuns maxim admis cerut pentru E1.



Deci se poate observa că, folosind maniera „tradițională” de lucru cu întreruperi există situații care nu pot fi tratate.

De aceea, în cazul sistemelor de operare în timp real, unde timpul de răspuns maxim admisibil (dead line time) este critic, trebuie folosit un alt sistem pentru administrarea evenimentelor recepționate.

Soluția este folosirea programării concurente. Cum a fost prezentat în laboratorul anterior, baza acestuia o constituie procesele (task-uri), funcții care se execută de regulă într-o buclă infinită și între care se face comutarea periodică de către un administrator de

task-uri. În mod normal fiecare task are ca obiect răspunsul la un eveniment. Comutarea între task-uri este realizată la intervale de timp suficiente de mici astfel încât să nu fie depășite timpurile de răspuns pentru fiecare din evenimente.

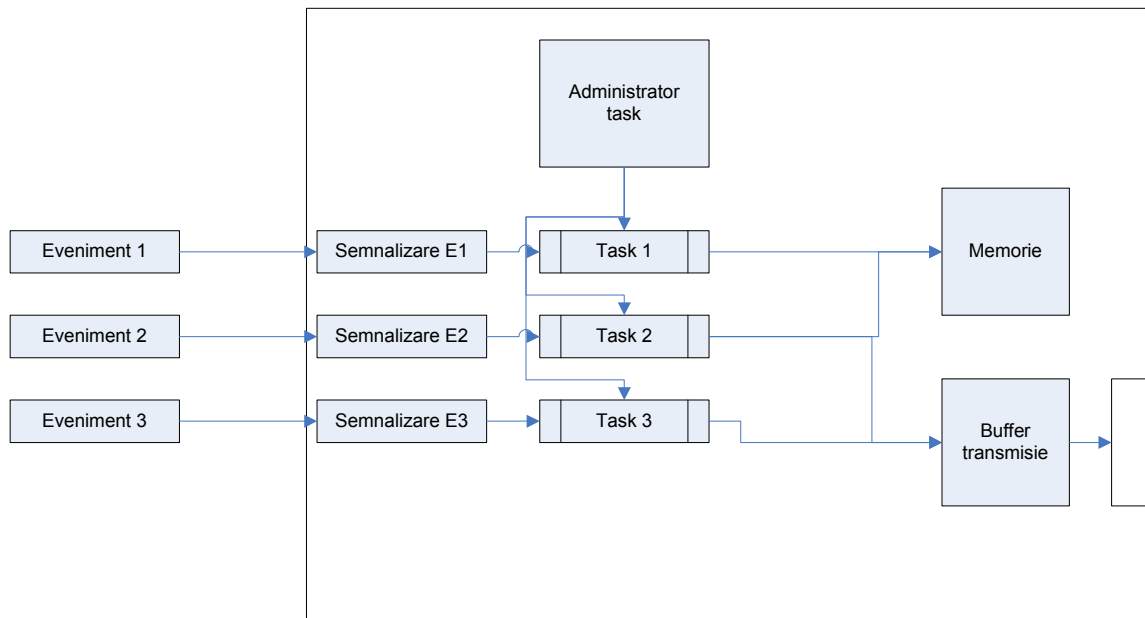
Pentru exemplificare este reluat cazul de mai sus: dacă avem doua evenimente E1 și E2 și avem pentru fiecare prevăzute două procese P1 și P2. În aceste condiții, ambele procese sunt prevăzute cu bucle de așteptare pentru evenimente. Să presupunem că apare evenimentul E1. Procesul P1 va detecta apariția acestui eveniment și va demara subrutina de tratare. Spre deosebire de situația anterioară, după un interval de timp se va comuta automat pe cel de-al doilea proces, după un interval de timp suficient de mic ales. În cazul în care evenimentul E2 apare, acesta va fi detectat de procesul P2 și se va intra în tratarea acestuia. Datorită sistemului de execuție a proceselor concurrent, răspunsul la evenimente poate fi furnizat acum în timp util.

3. Desfășurarea lucrării

Pentru a evidenția cele spuse mai sus se va utiliza mediul IAR împreună cu utilitarul IAR Make Applications și IAR CSpy.

În cele ce urmează se vor efectua următoarele:

- se vor folosi întreruperile pe portul paralel extern pentru a simula apariția unui eveniment;
- se vor construi administratorul de procese și procesele folosind sistemul time-sharing ca în laboratorul anterior;
- se va ilustra modul în care procesul răspunde la eveniment.
-



Pentru a putea lucra în acest laborator sunt necesare cunoștințele dobândite în laboratoarele anterioare! Anumite operații vor fi prezentate sumar, pentru detalieri consultați laboratoarele anterioare!!

Problema care va fi tratată este următoarea: se generează trei evenimente E1, E2 și E3. Acestea vor fi capturate pe pinii de port paralel ca întreruperi care apar pe acești pini. Se va construi un sistem care să răspundă la aceste evenimente.

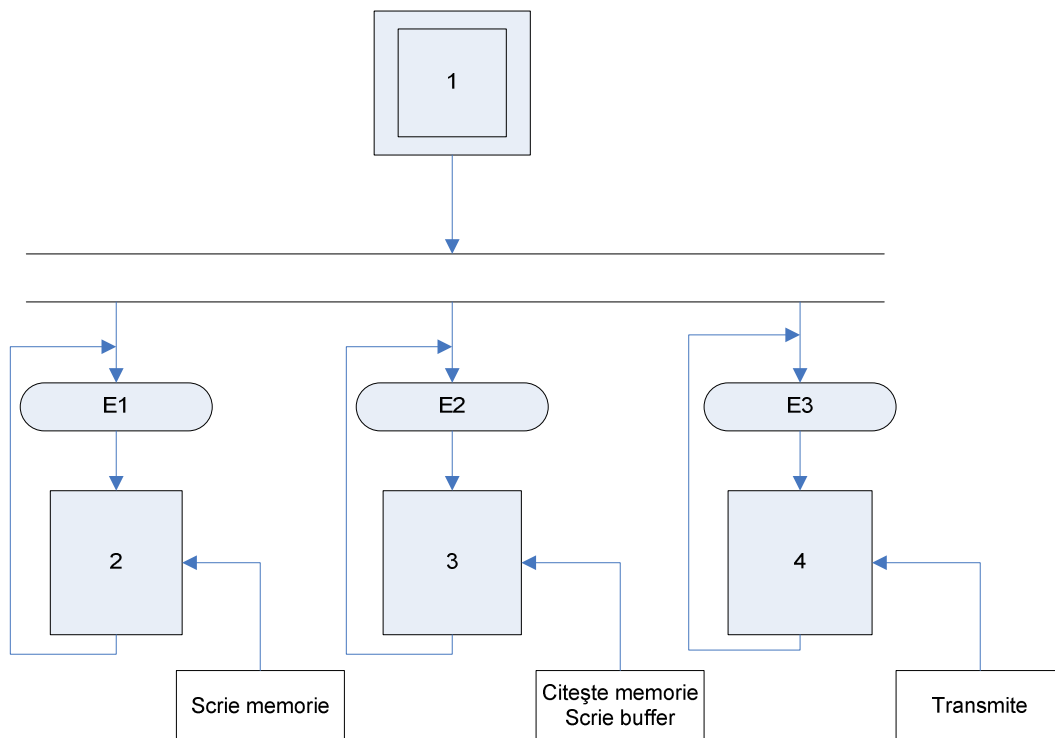
Sunt realizate 3 procese. Fiecare din aceste procese tratează evenimentul respectiv și va realiza contorizarea numărului de răspunsuri la eveniment.

Pentru exemplificare se consideră următoarele resurse și cerințe legate de evenimente și tratarea lor:

Resurse hardware:

- memoria de dimensiune 100 octeți definită ca un vector: `char memorie[100]`;
- buffer-ul de transmisie de dimensiune 100 octeți definit ca un vector `char buffer[100]`;
- canalul de comunicare serial UART0;
- canalul timer 0 ;
- pinii acționați pentru semnalizarea evenimentelor externe: INT0, INT1 și INT2;
- frecvența de lucru al microsistemului este de 16MHz.

Evenimentul 1: semnalizat de întreruperea INT0, declanșează depunerea unui număr, în sens crescător în memorie. Evenimentul 2, semnalizat de întreruperea INT1 va determina preluarea unui număr din memorie și aplicarea unei operații de XOR cu valoarea 0xa2 și salvarea acestuia în buffer. În fine, cel de-al treilea eveniment va determina transmiterea unui octet din buffer pe serială, cu viteza de 9600 bps și setarea 8,1,N (8 biți de date, 1 bit de stop și fără bit de paritate). În figura de mai jos este dată diagrama funcțională a sistemului, descrisă în GRAFCET.



Administratorul de procese realizează comutarea între procese la un interval de timp de 2 ms.

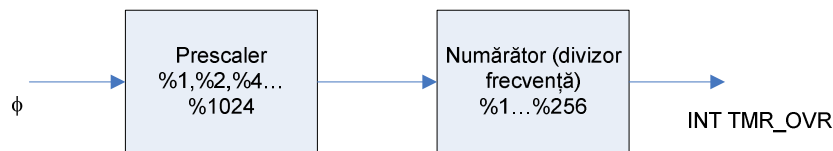
CONFIGURAREA RESURSELOR HARDWARE:

Pentru configurare se va folosi utilitarul IAR MakeApp. Reamintim că după ce se crează un proiect în MakeApp accesul la configurări se face prin acționarea butoanelor care corespund fiecărui periferic. În cazul de față configurările pentru canalul timer se fac prin apăsarea butonului TMR, pentru întreruperile externe prin butonul INTC iar pentru comunicația serială butonul USART. După setările făcute la fiecare periferic se va selecta Generate All Function iar în final se va compila. Atragem atenția că fișierele librărie generate în urma compilării se vor afla în directorul setat.

Resursele hardware folosite pentru această aplicație sunt, conform datelor de mai sus următoarele:

Canal timer 0

Va fi programat să genereze întreruperi la un interval de timp de 2ms deci cu frecvența de 500Hz . Având frecvența cuarțului de 16MHz se va realiza următoarea divizare:



$$16.000.000 / 1024 = 15.625 / 500 \sim 32.$$

Pentru o divizare cu 32 constanta de divizare este:

$$256 - 32 = 224 = 0xE0$$

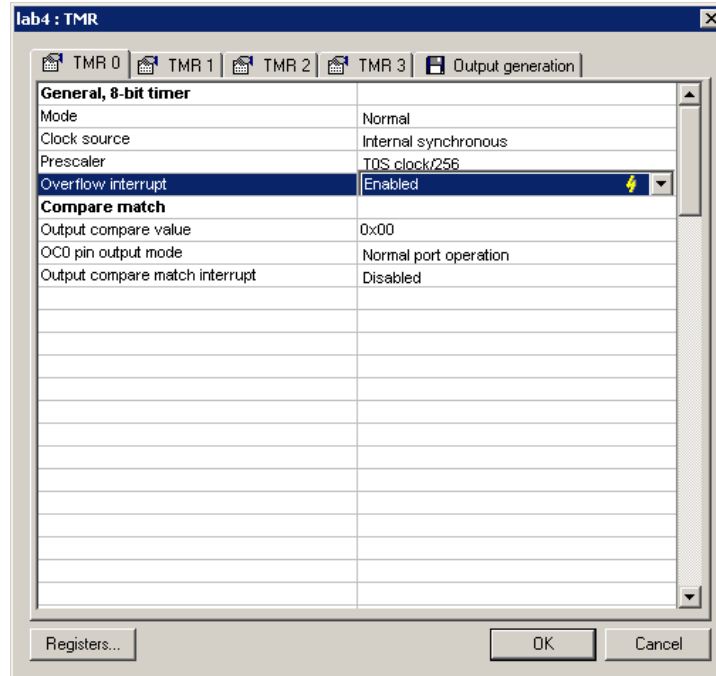
Programarea se va face utilizând utilitarul MakeApplication (butonul TMR din bara cu butoane de configurare).

Modul de lucru: normal;

Ceas: intern (de la ceasul microsistemului);

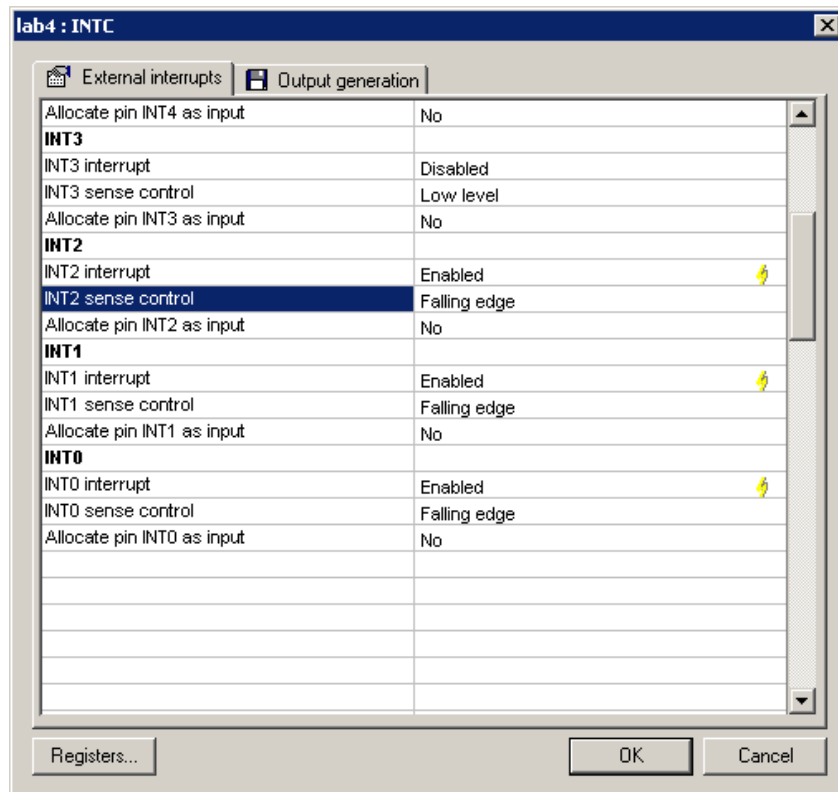
Prescaler: 256;

Întrerupere: activată.



Activarea întreruperilor externe:

Se vor activa întreruperile externe INT0, INT1 și INT2. Pentru toate se va activa ca front activ frontul descrescător.

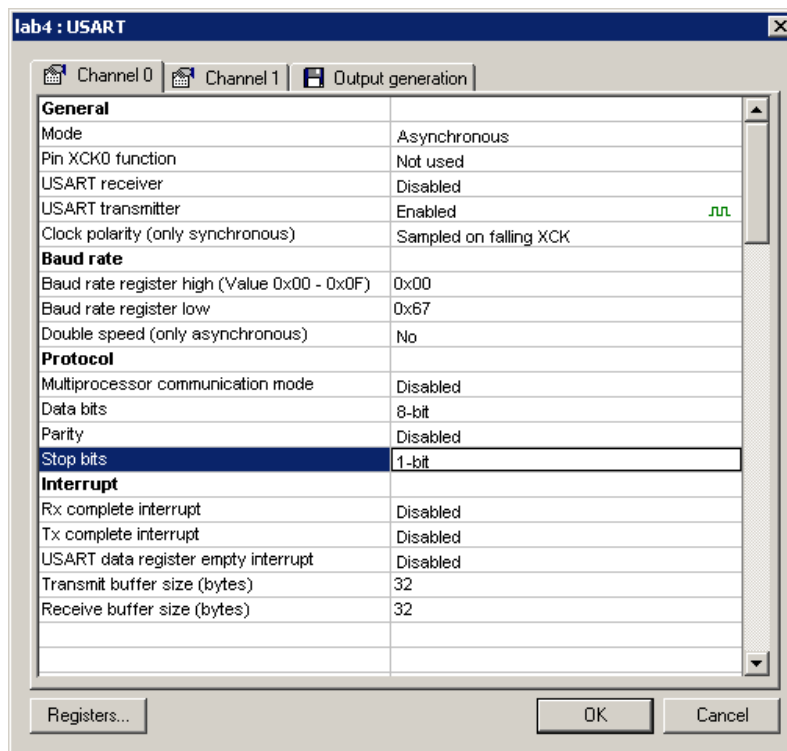


Pentru lucru cu canalul serial UART 0 se vor efectua următoarele configurații:
 Modul asincron.
 Activare transmisie.

Pentru setarea vitezei de comunicație va trebui scris registrul UBRR conform următorului calcul:

$$UBRR = \frac{f_{osc}}{16 \text{ Viteza}} - 1 = \frac{16.000.000}{16 \cdot 9600} - 1 \approx 103 = 0x67$$

8 biți de date
 Fără paritate
 1 bit de stop.



CONFIGURAREA SOFTWARE:

a. Variabile utilizate:

Fiecare eveniment va avea asociat o variabilă care va semnaliza faptul că a avut loc. Această variabilă va fi setată în interiorul subrutinei de tratare a întreruperi externe. Fiind trei evenimente, vor fi trei subrutine și deci vor fi trei variabile care semnalizează: e1, e2 și e3:

```
unsigned char e1=0,e2=0,e3=0;
```

În subrutine vor fi setate în 1 iar în momentul în care sunt tratate vor fi resetate (0).

Vor fi alocați vectorii pentru memorie și pentru bufer-ul de transmisie:

```
unsigned char memorie[100];//memorie accesata
```

```
unsigned char buffer[100];//bufer transmisie
```

Se vor declara variabilele index folosite pentru a determina care va fi următorul element în vector accesat. Vom avea variabila index pentru memorie:

```
unsigned char index_memorie=0;//index memorie
```

variabila index pentru bufferul de transmisie la scriere:

```
unsigned char index_buffer=0;//index buffer
```

și variabila index pentru bufferul de transmisie la citire:

```
unsigned char index_transmisie=0;//index pozitie transmise din buffer.
```

Va exista și un contor pentru task-ul curent necesar administratorului de task-uri.

```
unsigned char task_crt=0;//index task executat curent
```

Pentru a putea lucra cu aceste variabile globale, fără a fi nevoiți, în fiecare fișier din proiect, să declarăm variabilele însoțite de directiva „extern” s va proceda astfel:
Se creează un fișier numit **variabile.c** în care se vor declara toate variabilele de mai sus:

Fișierul variabile.c

```
//variabile
```

```
unsigned char e1=0,e2=0,e3=0;//indicatoare evenimente
```

```
unsigned char memorie[100];//memorie accesata
```

```
unsigned char buffer[100];//bufer transmisie
```

```
unsigned char index_memorie=0;//index memorie
```

```
unsigned char index_buffer=0;//index buffer
```

```
unsigned char index_transmisie=0;//index pozitie transmise din buffer
```

```
unsigned char task_crt=0;//index task executat curent
```

Acest fișier se va include în proiectul creat în IAR.

Se va crea un fișier numit **variabile.h** în care se vor include declarațiile variabilelor însoțite de directiva extern ca mai jos:

Fișierul variabile.h

```
extern unsigned char e1,e2,e3;//indicatoare evenimente
```

```
extern unsigned char memorie[100];//memorie accesata
```

```
extern unsigned char buffer[100];//bufer transmisie
```

```
extern unsigned char index_memorie;//index memorie
```

```
extern unsigned char index_buffer;//index buffer
```

```
extern unsigned char index_transmisie;//index pozitie transmisa din buffer
```

```
extern unsigned char task_crt;//index task executat curent
```

Pentru a putea folosi variabilele oriunde în proiect nu rămâne decât să declarăm, la începutul fiecărui fișier

```
#include "variabile.h"
```

2. Modificarea funcțiilor din librării:

De interes în acest proiect sunt următoarele fișiere generate de MakeApplication:

- ma_intc.c și ma_intc.h – fișiere cu funcții întreruperi externe;
- ma_tmr.c și ma_tmr.h – fișiere cu funcții întrerupere canal timer;
- ma_usart.c și ma_usart.h – fișiere cu funcții transmitere serială.

Reamintim că pentru a putea fi compilate, fișierele librărie cu extensia .c trebuie incluse în proiect și trebuie comentat în interiorul lor toate expresiile care încep cu ENTER_MA, EXIT_MA sau INSIDE_MA precum și incluziunea #include "usercode.h"

În plus în fișierul ma_intc.c se vor face următoarele modificări:

În subrutina de tratare a întreruperii externe pentru fiecare întrerupere în parte se va seta în 1 variabila indicator eveniment corespunzătoare: e1, e2 sau e3.

De exemplu, în subrutina de tratare a întreruperii externe INT0 avem:

Fișierul ma_intc.c - modificare

```
#include "variabile.h"
```

```
#pragma vector=MA_INT0_vect
```

```
__interrupt void MA_IntHandler_INT0_INTC( void )
```

```
{
    e1 = 1; //setare variabila eveniment
}
```

În fișierul `ma_tmr.c` unde se află declarată subrutina de tratare a întreruperii generate de canalul timer la un interval de 2 ms se va introduce codul pentru administratorul de task-uri.

Fișierul `ma_tmr.c` - modificare

```
#include "variabile.h"
#include "main.h"

#pragma vector=MA_TIMER0_OVF_vect
__interrupt void MA_IntHandler_OVF0_TMR( void )
{
    TCNT0 = 0xe0; //incarcare constata divizare pentru reluare numarare
    switch(task_crt) //interogare numar task curent
    {
        case 0: task1();
        break;
        case 1: task2();
        break;
        case 2: task3();
        break;
    }

    task_crt++; //incrementare numar task curent
    if(task_crt==3)
        task_crt=0;
}
```

3. Funcția principală și definirea task-urilor

La definirea task-urilor s-a ținut cont de cerințele aplicației, prezentate în diagrama GRAFCET. Task-urile sunt declarate ca funcții acolo unde a fost definită și funcția principală adică în fișierul `main.c`. Pentru a putea fi apelate de către administratorul de task-uri în `ma_tmr.c` s-a creat și un fișier header `main.h` în care s-au declarat funcțiile task:

Fișierul `main.c`

```
//incluziuni
#include "variabile.h"
```

```

#include "ma_usart.h"
#include "ma_tmr.h"
#include "ma_intc.h"
#include <inavr.h>
#include <stdio.h>

```

```

void task1(void)
{
    if(e1)//eveniment 1?
    {
        e1=0;
        if(index_memorie<100)//spatiu liber in memorie?
        {
            memorie[index_memorie]=index_memorie;//depunere data in memorie
            index_memorie++;
        }
    }
}

```

```

void task2(void)
{
    if(e2)//eveniment 2?
    {
        e2=0;
        if((index_buffer<100)&&(index_buffer<index_memorie))//spatiu liber in buffer?
        {
            buffer[index_buffer]=memorie[index_buffer] ^ 0xa5;//citire memorie depunere in
            buffer
            index_buffer++;
        }
    }
}

```

```

void task3(void)
{
    if(e3)//eveniment 3?
    {
        e3=0;
        if(index_buffer)//buffer transmisa contine ceva?
        {
            if((index_transmisie<100) && (index_transmisie<index_buffer))//mai sunt octeti de
            transmis?
            {
                printf("Transmite %d\n",buffer[index_transmisie]);//transmite octet
                index_transmisie++;
            }
        }
    }
}

```

```

    }
}
}

void main(void)
{

//initializari circuite periferice

    MA_Init_INTC();//initializare intreruperi externe

    MA_Init_TMR();//initializare canale timer
    MA_Start_TMR(0,1);//pornire canal timer 0

    MA_InitCh0_USART();//initializare canal USART 0

//activare intreruperi
    __enable_interrupt();

    for(;;);
}

```

Fișierul main.h

```

//declarare functii main
void task1(void);
void task2(void);
void task3(void);

```

Reamintim că pentru a nu fi nevoie de specificarea căii către fișierele header acestea trebuie să se găsească în același director cu fișierul main.c care le apelează.
La acest proiect ar trebui să avem următoarele fișiere în același director:

Nume fișier	Specificări	Sursă
Ma_intc.c	Trebuie inclus în proiect	Generat cu makeapp – modificat
Ma_intc.h		Generat cu makeapp
Ma_tmr.c	Trebuie inclus în proiect	Generat cu makeapp – modificat
Ma_tmr.h		Generat cu makeapp
Ma_usart.c	Trebuie inclus în proiect	Generat cu makeapp – modificat
Ma_usart.h		Generat cu makeapp
Ma_tgt.h	Utilizat de fișierele librărie	Generat cu makeapp
Ma_io.h	Utilizat de fișierele librărie	Generat cu makeapp
Main.c	Trebuie inclus în proiect	Creat
Main.h		Creat
Variabile.c	Trebuie inclus în proiect	Creat
Variabile.h		Creat

RULAREA APLICAȚIEI:

După compilare se va rula simulatorul. Înainte de a porni execuția programului se vor face mai multe setări.

În primul rând se va configura simulatorul să genereze întreruperi astfel (din meniu Simulator – Interrupts):

Întrerupere	Activation time	Repeat interval
INT0	100	100
INT1	200	200
INT2	300	300
TIMER0 OVF	10	10

Se va afișa fereastra Terminal I/O (din meniu View – Terminal I/O).

Se vor afișa în fereastra Watch (din meniu View – Watch) variabilele:

- memorie;
- buffer;
- index_memorie;
- index_buffer;
- task_crt.

Se va plasa un breakpoint la începutul subrutinei de la timer. Se va rula programul până la breakpoint apoi se va rula pas cu pas și se va observa incrementarea variabilei task_crt și comutare task-urilor deci cum lucrează administratorul de task-uri.

Se vor plasa breakpoint-uri în interiorul funcțiilor task, se va rula până la ele apoi pas cu pas pentru a se observa încărcarea memoriei și a buffer-ului, precum și afișarea în fereastra Terminal I/O .

Se va rula programul fără întrerupere apoi se va opri și se vor observa datele afișate în fereastră și starea variabilelor.

Aplicații, întrebări, probleme:

1. Care erau modificările care trebuiau făcute dacă se dorea o comutare între task-uri la un interval de 0.1ms?
2. Care este rolul variabilei task_crt?
3. De ce este necesară definirea a două variabile: index_buffer și index_transmisie pentru accesarea aceluiasi vector buffer? Există vreo situație în care se poate folosi doar una singură?
4. Să se modifice rata de probabilitate a apariției întreruperilor la 90% pentru INT0, 60% pentru INT1 și 40% pentru INT2. Care sunt modificările care apar la repornirea execuției? Ce se întâmplă cu timpul de execuție al task-urilor?
5. Să se modifice programul astfel încât când se ajunge la dimensiunea maximă (100) la memorie sau la buffer operațiile să fie reluate de la 0. În această situație ce se poate întâmpla dacă frecvența de apariție a evenimentului 1 este mult mai mare decât a evenimentului 2 și 3?
6. Să se proiecteze un sistem în timp real care monitorizează accesul într-o incintă cu grad sporit de pericolozitate. Incinta are 2 intrări fiecare prevăzută cu un

senzor de proximitate. În condițiile în care este detectată prezența unei persoane la una din intrări este declanșată o alarmă care suspendă procesul care are loc în incintă. Timpul maxim admisibil în care sistemul trebuie să răspundă în cazul în care primește o semnalizare de la senzori este de 0.5 secunde. Evenimente: semnalizare senzor ușă 1, semnalizare senzor ușă 2. ieșire: comandă declanșare sistem alarmă.

Indicație: Pentru implementarea acestei probleme se vor folosi întreruperile externe pe post de evenimente iar comanda va fi trimisă sub forma unui mesaj care va fi afișat în fereastra Terminal I/O. Intreruperile externe se vor declanșa doar o singură dată și nu repetat. De asemenea, la proiectarea administratolui task-urilor se va tine cont de timpul maxim admisibil de răspuns.